

1 septembrie 2023

1. Structura sistemelor de operare

a) Dati doua exemple de **organizare** a structurii sistemelor de operare.

Structura pe straturi (layered)

Monolithic

Microkernel

Modular

b) Explicati pe scurt **diferentele** dintre cele doua exemple de la punctul anterior.

Structura pe straturi – sistemul de operare separa functionalitatile in straturi, unde straturile de jos sunt din ce in ce mai aproape de hardware iar cele de sus cat mai aproape de user.

Monolithic – alcatuit dintr-un singur bloc de cod care contine toate functionalitatile sistemului, fara separare clara intre componente (ex Linux, Unix traditional).

Microkernel – impartit intre un nucleu minim care gestioneaza doar functiile esentiale. Restul functionalitatilor sunt implementate ca servicii in spatiul utilizatorului.

Modular – amestec intre monolithic si microkernel. Kernel-ul poate incarca si descarca module pe masura necesitate, combinatie de flexibilitate si performanta.

2. Exceptii si intreruperi

a) Ce sunt **exceptiile**? Dati 2 exemple.

Exceptiile = evenimente anormale care apar in timpule executiei unui program si necesita interventia sistemului de operare pentru a fi gestionate. Sunt generate de procesor atunci cand detecteaza o conditie de eroare sau o situatie speciala.

Impartirea la 0 = un proces incearca sa imparta un nr la 0.

Page Fault (Greseala de paginare) = un proces acceseaza o pagina de memorie care nu este inca in memoria fizica (RAM). Kernelul actioneaza prin incarcarea paginii necesare din memoria secundara (HDD,SSD, etc) in RAM.

Segmentation fault (Greseala de segmentare) = un proces incearca sa acceseze o zona de memorie nepermisa sau invalida.

b) Care e rolul **intreruperilor** in operarea sistemelor de calcul?

Permit sistemului de operare sa reactioneze la evenimente externe (ex. I/O, date primite din retea) fara ca procesorul sa fie blocat asteptand aceste evenimente. Ajuta la eficienta, interactivitate, gestionarea perifericelor.

c) Ce face sistemul de operare (**kernelul**) ca **raspuns la o intrerupere hardware**? Descrieti **mecanismul de tratare a intreruperilor**.

Intrerupere hardware = un dispozitiv (I/O, hard disk, retea) declanseaza o intrerupere. Procesorul si sistemul de operare raspund prin:

- 1) Realizeaza o intrerupere de sistem
- 2) Salveaza contextul curent (registrii, pointer de instructiuni, fisiere)
- 3) Identifica intreruperea (verifica tabelul vectorului de intreruperi)
- 4) Executa handler-ul de intrerupere (kernelul ruleaza rutina de tratare a intreruperii corespunzatoare)
- 5) Restaurareaza contextul la cel de dinainte de intrerupere

3. Planificarea proceselor.

a) Ce e **inversiunea de prioritati**?

Inversiunea de prioritati = atunci cand un proces cu prioritate mai mare trebuie sa astepte un proces cu prioritate mai mica, din cauza unei resurse partajate. E o problema de sincronizare ce poate duce la incetinirea executiei sau chiar deadlock in sistemele de operare cu planificare bazata pe prioritati.

b) Dati un **exemplu** de inversiune de prioritati.

Consideram prioritatea crescatoare: $4 > 1$

Avem 3 procese: P1 – prioritate 5, P2 – prioritate 3, P3 – prioritate 1.

P3 are blocata o resursa. P1 are nevoie de acea resursa si trebuie sa astepte ca P3 sa o elibereze. Intre timp, P2 incepe sa ruleze si blocheaza P3, deoarece P2 are prioritate mai mare decat P3, dar mai mica decat P1.

c) Prezantati pe scurt o **solutie** pentru rezolvarea problemei inversiunii de prioritati.

O solutie comuna – **Protocolul de mostenire a prioritatii** (Priority Inheritance Protocol).

Daca un proces P1 cu prioritate mare asteapta dupa o resursa blocata deja de un proces cu prioritate mai mica P2, atunci P2 mosteneste temporar prioritatea lui P1. Dupa ce resursa este eliberata de P2, acesta se intoarce la prioritatea initiala, iar acum P1 poate continua executia.

c*) Prezantati pe scurt o solutie pentru rezolvarea problemei inversiunii de prioritati folosind **Lottery Scheduling** (planificarea de tip loterie).

Lottery Scheduling atribuie fiecarui proces un numar de tichete proportional cu prioritatea sa. Pentru a rezolva problema inversiunii de prioritati se poate utiliza o tehnica de mostenire a numarului de bilete.

Cand un proces cu prioritate mare (deci, cu numar mare de bilete) este blocat, asteptand o resursa detinuta de un proces cu prioritate scazuta, se transfera temporar bilete de la procesul blocat catre cel care detine resursa.

Procesul care detine resursa „mosteneste” o parte din biletele proceselor cu prioritate superioara, sporindu-si astfel sansele de a fi selectat de scheduler. Acest lucru il ajuta sa termine sectiunea critica mai repede, si, astfel, sa elibereze resursa, reducand efectul inversiunii de prioritati.

d) Enumerati principalele **stari** in care se poate afla un **proces in executie** si explicati in ce conditii se face tranzitia intre aceste stari?

1. NEW (Nou) – procesul e creat. Intra in starea READY dupa ce sistemul il admite pentru executie.

2. READY (Gata de executie) – procesul e pregatit sa ruleze si asteapta alocarea CPU-ului. Trece in starea RUNNING cand scheduler-ul il selecteaza pentru executie. Poate reveni in starea READY daca e preemptat.

3. RUNNING (In executie) – procesul ruleaza efectiv pe CPU. Se poate muta in starea BLOCKED (sau WAITING) daca solicita operatii ce necesita asteptare. Poate fi preemptat de scheduler si reveni in starea READY.

4. BLOCKED/WAITING (Blocat/In asteptare) – asteapta un eveniment extern. Revine in starea READY cand evenimentul asteptat are loc.

5. TERMINATED (Terminat/Finalizat) – procesul si-a finalizat executia sau a fost intrerupt din cauza unei erori.

e) Care e semantica apelului sistem Unix **vfork** si de ce era necesar? Care sunt **limitarile** de utilizare (sau precautiile necesare) atunci cand se foloseste vfork?

vfork() – creeaza un nou proces (copil) care nu copiaza complet spatiul de adrese al parintelui, ci il imparte temporar. Pana cand copilul nu apeleaza `exec()` sau `exit()`, parintelui i se suspenda executia. In copil, valoarea returnata din `vfork()` este 0, iar in parinte este PID-ul copilului.

Necesar: crearea unui proces copil cu cost redus, evitand copierea intregului spatiu de memorie (precum in cazul `fork()`) atunci cand se stie ca copilul va apela imediat `exec()`.

Limitari:

- modificarea memoriei: copilul nu trebuie sa modifice variabilele sau datele din spatiul de memorie comun, deoarece sunt partajate cu parintele

- apeluri de sistem: nu se recomanda ca in copil sa se apeleze functii care pot bloca/modifica starea procesului parinte

- suspendarea parintelui: parintele ramane suspendat pana cand copilul finalizeaza executia (prin `exec()` sau `exit()`), deci un comportament incorect poate duce la blocarea intregului proces

f) Enumerati 2 **avantaje** ale **thread-urilor kernel** fata de **procese**.

1) Cost redus de creare si context switching – thread-urile kernel, care ruleaza in spatiul de kernel, partajeaza acelasi spatiu de adrese. Context switch-ul e mai rapid decat intre procese, unde se schimba si spatiul de memorie.

2) Comunicare si partajare de date mai eficienta – thread-urile partajeaza resurse (memorie, descriptor de fisiere, etc) in cadrul aceluias proces, facilitand sincronizarea si schimbul de informatii fara a recurge la mecanisme costisitoare de inter-proces.

g) Enumerati 2 **avantaje** ale **thread-urilor user** fata de cele kernel.

1) Implementare rapida si costuri reduse de schimbare a contextului – thread-urile gestionate la nivel de utilizator nu necesita apeluri sistem complexe pentru schimbarea contextului, ceea ce duce la comutari foarte rapide intre thread-uri.

2) Flexibilitate in gestionarea programarii – scheduler-ul la nivel de utilizator poate implementa politici de planificare personalizate, adaptate aplicatiei, fara a depinde de politicile generice ale sistemului de operare.

h) Enumerati doua **dezavantaje** ale **thread-urilor user**.

1) Lipsa de vizibilitate in kernel – kernelul nu e constient de existenta thread-urilor user, astfel daca un thread blocheaza o operatie de sistem (ex. apel de sistem blocant), intregul proces se blocheaza.

2) Dificultati in multiprocesoare – deoarece kernelul nu vede thread-urile user, alocarea lor pe mai multe procesoare nu este realizata automat, si limiteaza paralelismul real in executia lor.

h*) Enumerati doua **dezavantaje** ale **thread-urilor user** rezolvate de solutii cum ar fi scheduler activations.

1) Problema blocarii intregului proces – scheduler activations combina avantajele planificarii la nivel de kernel cu cea la nivel de user, permite kernelului sa intervina si sa programeze individual thread-urile user, astfel evitand situatia in care un apel de sistem blocant afecteaza toate thread-urile.

2) Lipsa paralelismului pe procesoare – kernelul aloca eficient thread-urile user pe mai multe procesoare, oferind o mapare dinamica si echilibrata, care permite exploatarea completa a resurselor hardware.

i) Care sunt cerintele necesare unei solutii corecte de partajare a resurselor?

1) Excluderea mutuala (Mutual Exclusion) – doar un singur proces poate intra in sectiunea critica la un moment dat.

2) Progres – daca niciun proces nu se afla in sectiunea critica si unul sau mai multe procese doresc sa intre, atunci alegerea unui proces care sa intre, atunci alegerea unui proces care sa intre in sectiunea critica nu poate fi blocata in mod indefinit.

3) Asteptare limitata (Bounded Waiting) – numarul de accesari succesive ale sectiunii critice de catre alte procese, astfel incat niciun proces care solicita accesul sa nu aiba parte de starvation (starvation).

j) Demonstrati ca data fiind o colectie de joburi cu timpi de rulare cunoscuti a priori, algoritmul de planificare Shortest Job First (SJF) este optimal in sensul in care planificarea obtinuta are cel mai mic timp mediu de asteptare dintre toate planificarile posibile pentru colectia de joburi data. (Indicatie: porniti de la formula timpului mediu de asteptare si minimizati functia respectiva).

k) Daca joburile care trebuie planificate nu sunt disponibile simultan (nu au toate acelasi timp de sosire in sistem), mai este optimal algoritmul de planificare Shortest Job First (SJF)? Justificati raspunsul.

l) O functie multithreaded pe care trebuie sa o scrieti are doua sectiuni critice: increment() incrementeaza in intreg si insert() insereaza o structura de date intr-un buffer partajat de lungime fixa. Primitivele de sincronizare disponibile pentru implementarea sectiunilor critice sunt spin-lock-uri si semafoare. Pentru fiecare dintre cele doua sectiuni critice, specificati:

- ce primitive de sincronizare ati utiliza pentru implementare in fiecare caz

- justificati alegerea facuta anterior pentru fiecare caz de sectiune in parte

4. Remote Procedure Call (RPC)

a) Ce este un apel de procedura la distanta (RPC)? Enumerati doua diferente fata de un apel de procedura obisnuit.

RPC = mecanism ce permite unui program sa invoce o procedura care se executa pe un alt calculator din retea, ca si cum ar fi o functie locala. Detaliile retelei (comunicare, prelucrarea parametrilor, transmiterea mesajelor, gestionarea erorilor) sunt abstractizare de sistemul RPC.

Diferente:

1) Comunicare si latentă – in RPC comunicarea se face prin retea, implicand costuri de transmisie si latentă, avand intarzieri; apelurile locale se efectueaza in cadrul aceluiasi proces sau memorie, fiind mult mai rapid si fara probleme de retea.

2) Prelucrarea parametrilor si locatiei – in RPC parametrii si datele trebuie convertiti intr-un format ce poate fi transmis peste retea si ulterior reconstruiti catre destinatie => complexitate suplimentara; in apelurile locale nu e nevoie de astfel de conversii, deoarece datele sunt deja disponibile in acelasi spatiu de memorie.

b) Ce este o **operatie idempotentă**? Dati un exemplu de operatie idempotentă si un exemplu de operatie care nu este idempotentă.

Operatie idempotentă = operatie care, efectuata de mai multe ori, e ca si cum ar fi efectuata o singura data. Repetarea operatiei nu modifica starea sistemului dincolo de efectul primei executii.

Operatie idempotentă: setarea unei valori; de exemplu $x=3$;

Operatie care nu e idempotentă: incrementarea; de exemplu $x++$;

c) Ce este **semantica RPC exactly once** (exact o singura data)? Explicati folosind un exemplu de ce este dificil sa se implementeze aceasta semantica.

RPC exactly once – indiferent de numarul de retransmisii sau erori de retea, apelul de procedura la distanta este executat o singura data pe serverul remote. Daca clientul retrimite cererea din cauza unor factori, sistemul se asigura ca procedura nu este executata de mai multe ori.

Dificultate: e dificil sa se distinga intre un mesaj pierdut si unul la care s-a pierdut confirmarea.

Exemplu: un client apeleaza o procedura care adauga un nou element intr-o baza de date. Daca raspunsul de confirmare se pierde, clientul poate retrimite cererea. Fara un mecanism de detectare a duplicatelor, serverul ar putea adauga elementul de doua ori, rezultand intr-o inserare duplicata.

Pentru implementarea „exactly once”, e necesara o gestionare complexa a starii, de exemplu atribuirea unui ID fiecarui apel si stocarea informatiilor despre apelurile deja executate, pentru a preveni efectele secundare ale executiilor repetate. Solutia implica si sincronizare si tehnici de recuperare in caz de esec.

5. Sisteme multiprocesor

a) Cum se **clasifica arhitecturile multiprocesor** din punctul de vedere al accesului la **memorie**? Care sunt diferentele intre aceste arhitecturi?

Clasificare in functie de accesul la memorie:

1) **UMA** (Uniform Memory Access) – toate procesele partajeaza aceeasi memorie fizica, iar timpul de acces la orice locatie de memorie este (aproximativ) egal pentru toate procesoarele. (ex. Sistemele multiprocesor simetrice traditionale - SMP). Accesul la memorie e uniform, iar coerenta cache-ului se poate realiza simplu, de obicei prin protocoale de tip „snoopy bus”.

2) **NUMA** (Non-Uniform Memory Access) – memoria e distribuita fizic in module asociate fiecarui procesor sau grup de procesoare. Timpul de acces la memoria locala (asezata pe procesorul din acelasi nod) este mai mic decat cel de acces la memoria aflata pe alte noduri (acces remote). (ex. Sistemele ccNUMA – cache-coherent NUMA – de obicei in servere mari). Se impune o arhitectura mai complexa pentru mentinerea coerentei datelor, iar algoritmi de planificare pot tine cont de locatia memoriei pentru optimizarea performantei.

Diferente: uniformitatea sau non-uniformitatea timpului de acces la memoria partajata; conectarea procesorului propriu-zis cu memoria acestuia, protocoalele de coerenta (UMA – „snoopy bus”, NUMA – bazate pe directoare sau alte mecanisme mai complexe)

b) Considerati un sistem multiprocesor simetric care foloseste un protocol write-invalidare. De ce e **TATAS (Test-and-test-and-set)** mai potrivita pentru implementarea sectiunilor critice decat **TAS (Test-and-set)** si in ce conditii?

TAS efectueaza operatii atomice de fiecare data cand un proces incearca sa achizitioneze lock-ul, generand scrieri constante care, intr-un protocol write-invalidare, provoaca invalidari ale cache-ului pe celelalte procesoare. TATAS realizeaza initial un test repetitiv (citind valoarea lock-ului din cache) fara a executa operatia atomica, pana cand detecteaza ca lock-ul este disponibil.

Conditii: nivel ridicat de concurenta pentru accesul la sectiunile critice. In astfel de conditii, TATAS reduce numarul de operatii atomice constisitoare si minimizeaza traficul pe BUS.

c) Ce se schimba daca se foloseste TATAS in conjunctie cu un protocol write-update?

Intr-un protocol write-update, atunci cand un proces scrie o valoare intr-o locatie partajata, noua valoare este actualizata catre toate copiile din cache ale celorlalte procesoare, fara a le invalida.

Intrucat beneficiul TATAS in protocolul write-invalidare era reducerea numarului de scrieri atomice tocmai din cauza aparitiei invalidarilor, acum, in write-update, este substantial.

Write-update transmite in mod efectiv noua valoare catre toate cache-urile, operatiile scrise atomic nu genereaza invalidari, ci actualizari. Totusi, TATAS realizeaza verificarile. Deci, traficul pe BUS va avea loc.

d) Ce este **sincronizarea wait-free** (lock-free)?

Sincronizarea lock-free – garanteaza ca, in orice moment, cel putin unul dintre thread-uri va face progres, chiar daca unele thread-uri pot intampina intarzieri. Sistemul nu poate intra intr-o stare de deadlock, insa nu se asigura ca fiecare thread isi finalizeaza operatia intr-un numar limitat de pasi. Se garanteaza ca fiecare thread isi va finaliza operatia intr-un numar finit (si de obicei, mic) de pasi, indiferent de comportamentul altor thread-uri.

e) Dati doua exemple de dezavantaje ale sincronizarii cu instructiuni hardware atomice (**lock-based**) pe care sincronizarea wait-free (**lock-free**) le rezolva?

1) Blocarea si posibilitatea de deadlock – algoritmi cu instructiuni hardware atomice (ex. TAS sau compare-and-swap) pot conduce la situatii in care un thread e blocat (ex. e preemptat in timp ce detine un lock). Lock-free – evita blocarea totala si permite fie ca sistemul in ansamblu sa faca progres, fie ca fiecare thread sa isi finalizeze operatia intr-un numar limitat de pasi, si deci elimina riscul de blocaj.

2) Probleme de inversiune a prioritatilor – lock-based, in care un thread cu prioritate mare e blocat de un thread cu prioritate scazuta ce detine lock-ul, si poate duce la intarzieri semnificative sau, din nou, deadlock. Lock-free elimina utilizarea lock-urilor, astfel ca niciun thread nu e blocat de altul, iar fiecare operatie pe date partajate e efectuata folosind instructiuni atomice si evita efectele adverse ale inversiunii de prioritati.

f) Dati un exemplu de instructiuni hardware care permit implementarea unei memorii tranzactionale restranse la dimensiunea unui cuvant si explicati pe scurt semantica lor.

Compare-And-Swap – instructiune hardware atomica care opereaza asupra unui singur cuvant din memorie.

6. Sincronizarea proceselor.

a) Ce este un **monitor**? Care sunt avantajele monitoarelor?

Monitor = mecanism **software** de sincronizare pentru accesul concurent la resurse partajate. Structura de date de nivel înalt. Incorpora datele partajate și operațiile pe care le pot efectua procesele asupra lor. Au funcții specifice definite în cadrul lor. Asigură excluderea reciprocă. Pot include variabile condiție.

Avantaje:

- 1) Simplitate și modularitate – codul e mai clar și mai ușor de scris prin comparație cu mutex-urile și semafoarele manuale
- 2) Excludere reciprocă automată – monitorul asigură implicit că doar un proces are acces la resursa partajată la un moment dat.
- 3) Facilități de așteptare și notificare – includ variabile condiție ce permit proceselor să aștepte anumite evenimente fără blocarea întregului sistem

b) Ce sunt **variabilele condiție** și cum se folosesc?

Variabila condiție = mecanism în monitoare ce permite unui proces să aștepte un anumit eveniment și să fie notificat atunci când acel eveniment are loc. Operații: wait (aștepta), signal (trezește un proces care aștepta pe variabilă), broadcast (trezește toate procesele care așteaptă pe variabilă).

Se declară cu „condition”, și se pune parametru în operațiile apelate.

*c) Ce sunt semafoarele?

Semafor = mecanism **software** de sincronizare pentru accesul concurent la resurse partajate. Întreg nenegativ împartășit între thread-uri. Are 2 operații – wait și signal. Wait verifică dacă alt proces e deja în zona sa critică. Signal semnalizează finalizarea procesului către celelalte procese. Nu pot avea loc 2 operații simultan.

*d) Care sunt limitările **alternanței stricte** ca metoda software de sincronizare între două procese?

Alternanța strictă presupune că cele două procese să alterneze cu strictețe intrarea în secțiunea critică, adică fiecare proces are o tură fixă.

Limitări:

1) Ineficienta in cazul unui proces inactiv – daca un proces nu doreste sa intre in sectiunea critica sau este blocat din alte motive, celalalt proces este fortat sa astepte, chiar daca sectiunea critica este libera.

2) Lipsa de flexibilitate – nu permite adaptarea la diferite ritmuri de executie. Daca un proces are nevoie de acces repetat si rapid, va fi tot timpul nevoit sa astepte „randul” sau prestabilit.

3) Nu asigura progres optim – chiar daca sectiunea critica e libera, un proces ar putea fi blocat pe nedrept din cauza mecanismului rigid de alternanta.

*e) Ce probleme ale strictei alternante rezolva **solutia lui Peterson**? Raman si probleme nerezolvate?

Solutia lui Peterson – algoritm clasic de sincronizare, destinat asigurarii excluderii mutuale pentru doua procese care concureaza pentru accesul la o sectiune critica.

1) Variabile Partajate: flag[2] – de tip bool, indica daca fiecare proces doreste sa intre in sectiunea critica; turn – indica ce proces are prioritate in caz de conflict.

2) Functionare: cand un proces doreste sa intre in sectiunea critica, ii seteaza flag la true si atribuie turn celuilalt proces. Apoi intra in bucla de asteptare in care verifica daca e randul celuilalt proces si vrea sa intre. Daca celalalt proces nu vrea sa intre sau daca e randul lui i, atunci iese din bucla de asteptare si intra in sectiunea critica.

Pro: excludere mutuala, progres si asteptare limitata (se bazeaza pe busy waiting, si poate fi inefficient)

*f) Ce problema apare atunci cand un **proces** aflat intr-un **monitor** apeleaza **operatie signal** asupra unei **variabile conditie**? Care sunt solutiile posibile?

Daca un proces apeleaza signal pe o variabila conditie in timp ce niciun alt proces nu apeleaza wait (adica nu se afla in asteptare pe acea conditie), semnasiul se pierde. Astfel poate duce la situatii in care un proces care ulterior va apela wait nu va fi trezit si va ramane blocat.

*g) Monitoarele in Java se implementeaza folosind cuvantul cheie synchronized. O metoda etichetata synchronized (ex public synchronized void

method()) se executa in regim de excludere mutuala fata de celelalte metode sincronizate ale aceluiasi obiect. De asemenea, limbajul implementeaza metodele wait si notify care atunci cand sunt apelate din interiorul unei metode sincronizate au comportamentul apelurilor wait si respectiv signal pentru variabile conditie. Implementati in java o clasa semaphore care atunci cand este instantiata are comportamentul unui semafor numarator (mai exact, implementati metodele up si down ale clasei semaphore care corespund comportamentului primitivelor down si up ale semafoarelor numarate).

```
public class Semaphore {
    private int value; // numărul de permise disponibile

    // Metoda "down" (sau wait) – scade contorul și blochează dacă nu sunt
    // permise disponibile
    public synchronized void down() {
        while (value == 0) { // Folosim "while" pentru a evita problemele legate de
            // trezire falsă
            try {
                wait();
            } catch (InterruptedException e) {
                // Restaurăm starea de întrerupere și ieșim din metodă
                Thread.currentThread().interrupt();
                return;
            }
        }
        // Permisul a fost obținut
        value--;
    }

    // Metoda "up" (sau signal) – crește contorul și notifică un thread care
    // așteaptă
    public synchronized void up() {
        value++;
        // Notificăm un singur thread aflat în așteptare
        notify();
    }
}
```

7. Deadlock

a) Care sunt **conditiile** necesare producerii **deadlock-ului**?

1) Excludere reciproca (Mutual Exclusion) – fiecare resursa este fie alocata unui singur proces, fie este disponibila, dar nu poate fi folosita simultan de mai multe procese.

2) Ocupare si asteptare (Hold and Wait) – un proces care detine deja o resursa poate solicita resurse suplimentare fara a elibera resursele pe care le are deja.

3) Fara preemptivitate (No Preemption) – o resursa alocata unui proces nu poate fi eliberata de sistemul de operare, ci trebuie voluntar de procesul care o detine.

4) Asteptare circulara (Circular Wait) – exista un ciclu intre procese si resurse ($p_1 \rightarrow r_1 \rightarrow p_2 \rightarrow r_2 \rightarrow \dots \rightarrow r_n \rightarrow p_1$)

b) Cate dintre conditiile de mai sus trebuie indeplinite pentru a se putea produce un deadlock?

Toate cele 4 conditii de mai sus trebuie indeplinite pentru producerea unui deadlock. La eliminarea oricareia dintre ele, sistemul de operare poate gasi o modalitate prin care resursele sa fie alocate proceselor, chiar daca o va face incoerent.

8. Gestiunea memoriei

a) Descrieti mecanismul generic de suport hardware pentru **protectia memoriei**.

Se bazeaza pe utilizarea unei unitati de management a memoriei (**MMU**), integrate in hardware, care are rolul de a traduce adresele logice generate de procese in adrese fizice reale. In acest proces, MMU-ul verifica, conform unor mecanisme de protectie, daca accesul la o anumita adresa este permis pentru procesul solicitant. Se previne, deci, ca un proces sa acceseze zone de memorie care nu ii sunt alocate sau care apartin altor procese, asigurand integritatea si securitatea sistemului

b) Ce este **swapping**-ul si la ce ajuta?

Swapping = tehnica de gestionare a memoritiei care permite mutarea temporara a unui proces din memoria RAM pe disc pentru a elibera resurse si a permite executia altor procese.

Creste gradul de multiprogramare si permite rularea mai multor procese decat incap in memoria RAM. Asigura ca procesele inactive sau in asteptare nu ocupa resursele de memorie necesare proceselor active.

Dezavantaj: poate aparea latenta mare din cauza vitezei reduce a discurilor prin comparatie cu memoria RAM.

c) Dati 3 exemple de **algoritmi de alocare dinamica a memoriei cu partitii fixe** (gestiunea „hole”-urilor ce apar in asemenea sisteme). Explicati pe scurt cum functioneaza cei 3 algoritmi alesi.

Partitii fixe -> memoria e impartita in blocuri de dimensiuni fixe, iar alocarea unui proces trebuie sa tina cont de spatiile libere ramase dupa eliberarea memoriei.

- 1) First Fit – procesul este alocat in primul „hole” suficient de mare pe care il gaseste (PRO: rapid; CON: poate lasa fragmente mici de memorie neutilizabile)
- 2) Best Fit – procesul e alocat in cea mai mica zona disponibila care e suficient de mare pt el (PRO: minimizeaza fragmentarea interna; CON: poate genera multe „hole”-uri mici => fragmentare externa)
- 3) Worst-Fit = procesul e alocat in cea mai mare zona disponibila (PRO: lasa un „hole” mare dupa alocare => pot incapea procese mari; CON: poate duce la risipa de memorie pe termen lung)

d) Ce este **fragmentarea memoriei** si de cate tipuri este? Explicati.

Fragmentarea memoriei = spatiul de memorie nu e utilizat eficient din cauza modului in care sunt alocate si eliberate resursele.

- 1) Fragmentare interna – un proces primeste un bloc de memorie mai mare decat are nevoie, iar memoria nefolosita din acel bloc nu poate fi reutilizabila.
- 2) Fragmentare externa – memoria libera este impartita in bucati care nu pot fi folosite eficient de procesele care au nevoie de alocare continua.

e) Cum se face **transformarea adresei logice** in **adresa fizica** in sistemele de gestiune a memoriei cu **paginare**? (E suficient sa explicati mecanismul de suport hardware si cu ajutorul unei scheme).

Unitatea de Management a Memoriei (MMU) transforma adresele logice in adrese fizice utilizand tabele de pagini.

Schema:

- 1) Procesul genereaza o adresa logica (nr paginii p; offset-ul d – pozitia in pagina)
- 2) Adresa e convertita astfel: se cauta nr paginii (p), apoi cadrul (frame), iar offset-ul ramane acelasi. Formula:

Adresa fizica = Baza cadrului (Frame base) + Offset (d)

f) Rezolva paginarea vreuna dintre problemele fragmentarii memoriei? Explicati.

Paginarea rezolva problema fragmentarii externe, dar nu elimina complet fragmentarea interna.

Fragmentarea externa e eliminata deoarece paginarea permite ca procesul sa fie stocat in mai multe cadre (frame-uri) separate din memorie.

Fragmentarea interna poate aparea deoarece ultima pagina a unui proces poate fi partial umpluta (de ex un proces de 10.5 KB va folosi 3 pagini de cate 4KB – 2 intregi si din a 3-a vor ramane 1.5 KB neutilizati)

g) Descrieti operatia unui **TLB (Translation Look-aside Buffer)**. Care sunt implicatiile folosirii TLB asupra context-switching-ului si ce solutii se pot folosi?

Translation Look-aside Buffer (TLB) – memorie cache specializata in hardware care stocheaza recentul set de traduceri de la adrese liniare (sau virtuale) la adrese fizice, pentru a accelera procesul de traducere a adreselor. Cand procesorul genereaza o adresa liniara, TLB-ul este verificat pentru a gasi rapid traducerea corespunzatoare si evita accesul la tabelele de pagini din memoria principala.

1) TLB Hit – procesorul cauta in TLB adresa liniara

2) TLB Miss – daca traducerea nu e in TLB, procesorul acceseaza tabela de pagini pentru a obtine traducerea. Apoi, e stocata in TLB pentru accesarile viitoare

Problema: la un context switch, TLB-ul contine intrari care corespund spatiului de adrese al procesului anterior care trebuie golit. Dar golirea acestuia duce la multe TLB Miss-uri pentru noup proces.

Solutii:

- utilizarea Address Space Identifier (ASID) – fiecare intrare din TLB e etocjetata cu un ID care permite pastrarea intrarilor pentru mai multe procese simultan. La context switch, intrarile relevante raman valabile, deoarece ASID-ul noului proces e diferit, iar cele din TLB pot fi reutilizate fara a fi golite

- reload-ul incremental sau mentinerea unui TLB partajat intre doua procese (in sistemele cu anumite modele de securitate)

h) Ce e o **tabela inversata de pagini** si cand se foloseste? Ce se intampla cand exista pagini partajate in sistem (mentionati si solutii posibile)?

Tabela inversata de pagini = structura de date alternativa la tabelele de pagini clasice. In loc sa existe o tabela de pagini pentru fiecare proces (unde fiecare intrare corespunde unei pagini virtuale), tabela inversata are o intrare pentru fiecare cadru de pagina din memoria fizica.

Se foloseste – sisteme cu spatii de adrese foarte mari sau in sisteme cu multe procese, pentru reducerea dimensiunii totale a tabelelor de pagini. O singura tabela inversata, de dimensiunea numarului de cadre de pagina din memoria fizica, poate fi mult mai compacta decat tavelele separate per proces.

Cand exista pagini partajate – acelasi cadru de pagina poate fi mapat in spatiile de adrese ale mai multor procese. O tabela inversata standard, care are o singura intrare per cadru, trebuie sa gestioneze multiple asocieri (mai multe perechi proces/pagina) pentru acelasi cadru.

Solutii:

- intrari multiple sau liste – de exemplu fiecare intrare sa contina o lista de perechi <proces, adresa virtuala>

- structuri auxiliare – se pot folosi tabele suplimentare sau mecanisme de referentiere pentru a gestiona partajarea paginilor

9. Memoria virtuala

a) Cum functioneaza **paginarea la cerere** (demanding paging)?

Paginarea la cerere = tehnica in care paginile din spatiul de adrese logic al unui proces nu sunt incarcate in memoria fizica decat atunci cand sunt necesare (adica „la cerere”).

Procesul incepe executia cu o parte din imagini marcate ca „neincarcate”. Cand acceseaza o pagina care nu se afla in memorie, se genereaza un page fault. Kernelul si MMU-ul intercepteaza evenimentul si incarca pagina necesara din stocarea secundata (de obicei de pe disc) intr-un frame liber din memoria RAM. Dupa ce e incarcata pagina si tabelul de pagini e actualizat, executia procesului este reluata de la instructiunea care a generat page fault-ul.

b) Care sunt **avantajele** folosirii **paginarii la cerere**?

1) Eficienta memoriei – se incarca in memorie doar paginile care sunt efectiv utilizate, se reduce deci consumul de RAM si permite rularea unor programe mai mari decat memoria fizica disponibila.

2) Multiprogramare sporita – permite ca mai multe procese sa coexiste in memorie, deoarece nu toate paginile unui proces trebuie incarcate simultan.

3) Timp de pornire redus – procesul poate incepe executia mai rapid, fara a astepta incarcarea completa a intregului sau spatiu de adrese.

4) Flexibilitate – sistemul poate optimiza gestionarea memoriei prin retinerea in RAM a celor mai frecvent accesate pagini si descarcarea celor mai putin utilizate.

c) Ce este un **page-fault** si cum este tratat de catre sistemul de operare (kernel)?

Page fault = exceptie generata atunci cand un proces incearca sa acceseze o pagina de memorie care nu se afla in RAM sau pentru care nu are drepturi de acces (de exemplu una marcata ca „nevalida”).

Tratarea page fault-ului:

1) Detectarea – procesorul, prin MMU, detecteaza ca adresa accesata nu se afla in memoria fizica (sau accesul ilegal) si declanseaza o exceptie de tip page fault.

2) Suspendarea procesului – kernelul intra in modul de tratare a page fault-urilor si suspenda executia procesului care a generat exceptia.

3) Determinarea cauzei – kernelul verifica daca page fault-ul vine dintr-o cerere legitima (si pagina trebuie incarcata) sau reprezinta o eroare (ex. acces ilegal). In cazul unei erori, procesul poate fi terminat.

4) Alocarea unui frame – daca pagina trebuie incarcata, kernelul identifica un frame liber din memoria RAM sau selecteaza o pagina candidata pentru inlocuire.

5) Incarcarea paginii – pagina solocitata este citita de pe disc si plasata in frame-ul ales.

6) Actualizarea tabelor de pagini – se actualizeaza tabela de pagini astfel incat adresa logica a procesului sa fie corect mapata la noua adresa fizica.

7) Reluarea executiei – procesul suspendat este reluat, iar instructiunea care a cauzat page fault-ul este reexecutata, de aceasta data cu pagina disponibila in memorie.

d) Care este **formula timpului efectiv de acces la memorie (Effective Acces Time, EAT)** in sisteme cu paginare la cerere, dandu-se rata p ($0 \leq p \leq 1$) a page-fault-urilor?

$EAT = (1-p) * ma + p * (pf + ma)$, unde

ma = timpul mediu de acces la memorie (fara page-fault)

pf = penalizarea (timpul de servire a unui page-fault) – timpul de transfer al paginii de pe disc + alte operatii

e) Ce este tehnica de **Copy-On-Write (COW)** si la ce foloseste?

Copy-On-Write (COW) = folosita in gestionarea memoriei, in special la operatiile de copiere a proceselor (de ex in cazul apelului `fork()`). Asigura coerenta memoriei si pastrarea acesteia pe parcursul unor procese ce folosesc aceeasi zona de memorie.

La initierea unui `fork()`, parintele si copilul impart acelasi spatiu de memorie, ambele setate la read-only. Cand oricare dintre ele incearca sa scrie intr-o pagina comuna, se declanseaza o copie a acelei pagini (copy-on-write), astfel incat modificarile vor afecta doar procesul care a cerut scrierea.

Avantaje: economie de memorie (nu se copiaza intregul spatiu de memorie, ci doar paginile modificate efectiv), performanta imbunatatita (reduce timpul necesar pentru crearea unui nou proces, deoarece copierea completa a memoriei se face doar cand e necesara).

f) Cate tipuri de **algoritmi de inlocuire a paginilor** exista in functie de **setul de frame-uri din care se alege victima** (frame-ul care contine pagina ce va fi evacuata din memorie)? Explicati pe scurt consecintele alegerii fiecaruia dintre algoritmi.

2 abordari/tipuri:

1) Algoritmi de inlocuire globala (Global Replacement) – victima poate fi selectata din totalitatea paginilor aflate in memorie, indiferent de procesul caruia ii apartin

Pro: permite o mai buna utilizare globala a memoriei, deoarece se poate alege pagina cea mai putin utilizata din intregul sistem.

Contra: poate conduce la starvation pentru anumite procese, deoarece paginile unui proces pot fi evacuate chiar daca acel proces are nevoie intensa de memorie.

2) Algoritmi de inlocuire locala (Local Replacement) – victima este selectata doar din setul de pagini care apartin procesului ce a generat page fault-ul.

Pro: asigura alocarea mai echitabila a memoriei intre procese si previne ca un proces sa fie afectat excesiv de evacuarea paginilor sale.

Contra: poate duce la o utilizare suboptima a memoriei la nivel global, deoarece unele procese pot avea memorie alocata, dar neutilizata, in timp ce altele sufera page fault-uri frecvente.

10. Mecanismul de segmentare/paginare la Intel IA-32 (arhitecturi pe 32 de biti)

a) Explicati schema generala de transformare a adresei logice/virtuale in cea fizica.

1) Segmentare – adresa logica e o pereche (selector de segment, offset). Segmentul reprezinta un descriptor ce contine baza segmentului. Se obtine astfel adresa liniara (daca paginarea e activata) sau direct adresa fizica, altfel.

2) Paginare – daca paginarea e activata, adresa liniara e apoi tradusa in adresa fizica.

b) Cum se obtine adresa liniara din adresa logica?

Adresa logica (selector de segment, offset).

Adresa liniara = Baza segmentului + Offset

c) Cum se obtine adresa fizica din adresa liniara?

Daca paginarea nu e activata:

Adresa fizica = Adresa liniara

Daca paginarea e activata:

Adresa liniara are: indexul directorului de pagini; indexul tabelului de pagini, offset-ul in pagina.

Registrul CR3 contine adresa de baza a directorului de baza. Se indica apoi locatia tabelului de pagini. Din tabelul de bagini, indexul e folosit pentru a gasi intrarea paginii -> contine baza cadrului de pagina din memoria fizica.

Adresa fizica = Baza cadrului de pagina + Offset

11. Sisteme de fisiere/stocare a datelor

a) Ce este un **File Control Block (FCB)**? Ce tip de informatii contine?

File Control Block (FCB) = structura de date interna sistemului de fisiere care contine informatii esentiale despre acesta.

Informatiile continute: numele (sau un ID), atributele (drepturi de acces, tipul, permisiuni), dimensiunea, timpii de creare modificare accesare, locatia pe disc (pointeri sau blocuri), informatii de gestiune (starea, referinte pentru deschideri multiple).

b) Ce este un **buffer cache**? Dar un **page cache**? Ce problema apare cand ambele cache-uri exista in sistem si care ar fi o solutie posibila?

Buffer cache = zona de memorie pentru stocarea temporara a blocurilor de date citite de pe disc sau destinate scrierii pe disc. Ajuta la reducerea numarului de operatii I/O prin reutilizarea datelor deja citite sau pregatite pentru scriere.

Page cache = zona de memorie folosita de sistemul de gestionare a memoriei pentru a stoca pagini din spatiul de adrese virtuale, inclusiv pagini citite din fisiere sau biblioteci. Permite accesul rapid la datele stocate pe disc prin intermediul memoriei RAM.

Daca exista amandoua – e posibil ca datele sa fie duplicate (stocate in ambele cache-uri), poate => ineficienta in utilizarea memoriei, sau inconsistente (daca cele 2 cache-uri nu sunt sincronizate corespunzator)

Solutie: unificarea lor intr-o singura zona de cache, sistemul va gestiona o singura copie a datelor in memorie.

c) Care sunt componentele principale ale **timpului de acces** la date pentru **discuri clasice organizate** pe cilindri, piste si sectoare? Detaliati raspunsul.

Timpul total de acces la date al unui disc clasic e alcatuit din:

1) Timpul de cautare (seek time) – timpul pt capul de citire/scriere sa se miste de la o pista/cilindru la altul

2) Latenta de rotatie (rotational latency) – asteptarea ca sectorul dorit sa se alinieze cu capul, datorita rotatiei discului

3) Timpul de transfer (transfer time) – timpul efectiv de citire/scriere a datelor din sectorul dorit

d) Care componenta dintre cele de la punctul anterior poate fi optimizata folosind **algoritmi de planificare de disc**?

Timpul de cautare – algoritmi de planificare a discului (SSTF, SCAN, C-LOOK, etc) reduc miscarile capului intre cererile de acces si, astfel, imbunatatesc performanta.

e) Ce e un **i-node** Unix si la ce se foloseste?

i-node = structura de date din sistemele de fisiere Unix care stocheaza informatii despre un fisier, cu exceptia numelui sau.

Informatii: dimensiunea, permisiuni si drepturi de acces, timpi (creare, modificare, acces), numarul de legaturi (link-uri) catre fisier, pointeri catre blocurile de date (directe si indirecte) unde sunt stocate efectiv datele fisierului

f) Ce este un **v-node** si la ce se foloseste?

V-node (virtual node) = abstractie folosita de stratul Virtual File System (VFS) al kernelului pentru a reprezenta un obiect de fisier indiferent de sistemul de fisiere subadiacente. Permite kernelului sa gestioneze fisiere din diferite sisteme de fisiere intr-un mod uniform, ascunzand detaliile specifice ale fiecarui sistem.

g) Un sistem de fisiere Unix System V (s5fs) foloseste blocuri de dimensiune 4 KB si pointeri catre blocuri pe 4 octeti. Cat de mare este cel mai mare fisier pe care il poate stoca acest sistem de fisiere? (indicatie: folositi structura inode-ului)

Blocuri de 4 KB

Fiecare pointer = 4 bytes

i-node -> array de pointeri -> primii 7 directi si ultimul indirect

7 blocuri indirecte => $7 \times 4\text{KB} = 28\text{KB}$

1 bloc indirect => $4096 \text{ octeti} / 4 = 1024$ pointeri, fiecare pointer refera un bloc de 4KB

$1024 \times 4 = 4096 \text{ KB}$

Adaug blocurile directe: $28 + 4096 = 4124 \text{ KB}$

h) Un disk are 40 de cilindri. Fie o secventa de citiri care implica cilindrii 11, 1, 36, 16, 34, 9, 12 in aceasta ordine. Comparati timpul de cautare (seek time) al urmatorilor algoritmi: SSTF, SCAN (algoritmul liftului), C-LOOK. Exprimati timpul de cautare in termenii numarului de cilindri sariti de bratul de citire al discului pentru a satisface secventa de accese.

-ALG DE FACUT PE FOAIE

12. Solutia problemei readers/writers (cititori/scriitori) prezentata favorizeaza cititorii. Explicati in ce sens e vorba de favorizare si gasiti o solutie care favorizeaza scriitorii. Prezentrati pseudocodul acestei solutii folosind semafoare.

Solutia clasica – data ce un cititor a inceput sa citeasca, alti cititori pot intra in sectiunea critica chiar daca exista un scriitor care asteapta. Asta inseamna ca, daca un flux continuu de cititori soseste, scriitorul poate ramane blocat pentru perioade lungi de timp (starvation).

Pentru a favoriza scriitorii, solutia trebuie sa blocheze intrarea de noi cititori cand exista cel putin un scriitor care asteapta. Odata ce ultimul cititor activ isi termina operatia, scriitorul care asteapta poate obtine imediat accesul la resursa partajata.

13. Problema cu memorie:

Un PC are 512 MB RAM, un microprocesor pe 47 de biti, pagini de 16 KB si foloseste PTE-uri (intrati in tabela de pagini) de 8 octeti. Un program care ruleaza pe acest computer are 10 KB de text si date incarcate la baza spatiului de adrese si 5 KB de stiva la capatul superior al spatiului de adrese.

a) Cat de mare este o tabela liniara de pagini care mapeaza programul in RAM?

b) Trasati o diagrama (o schema) care mapeaza acest program in memoria RAM folosind o tabela de pagini pe 3 niveluri care foloseste un numar egal de biti de adresa pentru fiecare nivel. Cat spatiu ocupa aceasta tabela de pagini?

c) Descrieti o posibila implementare pentru tabela de pagini de la punctul (a) si calculati spatiul de memorie fizica necesar stocarii acestei tabele.